

Computer Graphics

Part 1: Foundations of Computer Graphics

1.1 What is Computer Graphics?

- Computer Graphics is the use of computers to create and manipulate pictures on a display device.
- It comprises software techniques to create, store, modify, and represent pictures.

1.2 Applications of Computer Graphics

- Entertainment
- Design and Visualization
- Education and Training
- Medical Imaging and Healthcare
- Scientific Research and Data Visualization

1.3 Interactive vs. Passive Graphics

Interactive Graphics

Involve real-time user interaction with computer-generated content. Users can manipulate, navigate, and modify visual elements dynamically, with immediate feedback.

- Video Games — control characters, explore worlds, interact with objects.
- CAD Software — manipulate 3D models, make design changes, view from different perspectives.
- VR Environments — immersive interaction with virtual objects using motion controllers.
- Data Visualization — explore datasets via charts, graphs, and interactive maps.

Passive Graphics

Involve display of pre-rendered or static visual content without user interaction — the user observes but cannot modify it in real time.

- Films & Television — pre-rendered VFX, animation, CGI.
- Printed Materials — books, magazines, posters, advertisements.
- Pre-rendered Animations — storytelling, educational content, promotional videos.

Part 2: Graphic Systems & Display Devices

2.1 Display Processor

An interpreter/piece of hardware that converts display processor code into pictures. It has four main parts:

1. Display File Memory — generation & identification of graphic entities.
2. Display Controller — handles interrupts, maintains timing, interprets instructions.
3. Display Generator — generates characters and curves.
4. Display Console — contains the CRT, light pen/keyboard, and deflection system.

2.2 Raster Scan System

A method of generating images by scanning an electron beam across the screen in a systematic pattern (left–right, then down), mainly associated with CRT monitors.

Raster Display System with a Display Processor — Components

5. Display Processor — generates signals controlling the electron beam.

6. Frame Buffer — memory storing image data for the current frame; each location = one pixel.
7. Raster Scan System — scans the beam in horizontal lines (raster grid) across the screen.
8. Display Controller — synchronizes raster scan timing with image data generation.
9. Digital-to-Analog Converter (DAC) — converts digital frame-buffer values into analog beam-intensity voltages.
10. Beam Deflection System — horizontal & vertical coils that steer the electron beam.
11. Pixel Illumination — beam excites phosphor dots, creating visible pixels.

2.3 Display Devices

- Refresh Cathode Ray Tube (CRT)
- Random Scan and Raster Scan
- Color CRT Monitors
- Direct View Storage Tubes (DVST)
- Flat Panel Display
- Lookup Table (LUT)

Refresh CRT

Scans an electron beam across a phosphor-coated screen using raster scanning, refreshing the display many times per second.

Random Scan vs Raster Scan

- Raster scan: beam systematically scans left → right, top → bottom, forming a grid of pixels.
- Random scan: beam is directed only to specific points/lines as needed (not a fixed raster pattern).

Color CRT Monitors

Use red, green and blue phosphors with multiple electron guns and shadow masks for precise color alignment. Two approaches:

i) Beam Penetration Method

- Used with random-scan monitors; screen coated with red & green phosphor layers.
- Color depends on how far the beam penetrates — produces only 4 colors: red, green, orange, yellow.
- Advantage: inexpensive. Disadvantage: only 4 colors, lower picture quality.

ii) Shadow-Mask Method

Uses a perforated metal mask behind the phosphor screen to ensure each electron gun (R, G, B) excites only its matching phosphor dot, giving a much wider color range than beam penetration.

Direct View Storage Tubes (DVST)

- Uses random-scan approach; the screen itself retains (stores) the image so no refreshing is needed.
- Primary gun stores the picture pattern; flood/secondary gun maintains picture display.
- Advantages: no refreshing needed, high resolution, low cost.
- Disadvantages: cannot selectively erase part of a picture; not suitable for dynamic graphics; slow to modify.

Flat Panel Display

- Reduced volume, weight, and power vs. CRT (e.g., small TVs, calculators, laptops).
- Emissive displays — convert electrical energy directly into light (Plasma Panel, thin-film electroluminescent, LED).

- Non-emissive displays — use optical effects on external light (e.g., LCD).

Plasma Panel Display (Gas-Discharge Display)

An array of small fluorescent lights. Components: Cathode (delivers negative voltage), Anode (delivers positive voltage), Fluorescent cells (gas that emits light when voltage applied), Glass plates (act as capacitors).

- Advantages: high resolution, large screen size possible, low volume/weight, flicker-free.
- Disadvantages: complex wiring/addressing (note: some sources also list resolution limits).

LCD (Liquid Crystal Display)

Produces a picture by passing polarized light through liquid-crystal material sandwiched between two perpendicular glass plates with row/column conductors; pixel position = intersection of conductors.

- Advantages: low power consumption, small size, low cost.
- Disadvantages: temperature-dependent (0–70°C), low contrast (no self-emission), limited color capability, lower resolution than CRT.

2.4 Lookup Table (LUT)

Maps digital color values to specific display colors. In CRT displays it converts frame-buffer digital values into analog beam-intensity voltages per color channel (R, G, B). In modern displays, used for color correction, gamma correction, and calibration.

2.5 Cathode Ray Tube (CRT) — Components

12. Electron Gun — heating filament (heater) + cathode; produces and focuses the electron beam.
13. Control Electrode — turns the electron beam on/off.
14. Focusing System — focuses electrons into a narrow beam for a clear picture.
15. Deflection Yoke — creates electric/magnetic fields to steer the beam, linked to the sweep/scan generator.
16. Phosphorus-coated Screen — glows (phosphoresces) when struck by the high-energy electron beam.

2.6 Random Scan Display

- Beam acts like a pencil, drawing straight-line segments defined by (x, y) endpoints.
- Cycles back and redraws all lines 30–60 times per second.
- Refresh rate depends on the number of lines; picture definition stored as line-drawing commands in a refresh display file.
- A pen plotter is a classic example of a random-scan device.
- Refresh delayed to keep the rate ≤ 60 fps to avoid phosphor burn.

Advantages / Disadvantages

- Advantages: higher resolution than raster; smooth line drawing; less memory required.
- Disadvantages: cannot draw realistic shaded images; limited color.

2.7 Raster Scan Display

- Controls pixel intensity within a rectangular raster; on/off pixel info stored in the refresh/frame buffer.
- Household TVs use raster scan; refresh rate 60–80 fps.
- Frame buffer = raster/bit map; positions are called picture elements (pixels).
- Beam refresh: horizontal retrace (end of each line) and vertical retrace (bottom-right back to top-left).

Scanning Types

- Interlaced Scanning — traces each line top to bottom in two passes (odd lines, then even lines); can cause fading/flicker; typical refresh 60 fps.
- Non-Interlaced (Progressive) Scanning — traces all lines in one pass sequentially; typical refresh 30 fps but can flicker.

Part 3: Scan Conversion

3.1 Scan Conversion — Definition

The process of representing continuous graphics objects as a collection of discrete pixels, each of which is either on or off. The video display circuitry converts binary values (0/1) into pixel off/on information.

Advantages of Scan-Conversion Algorithms

- Generate graphics objects faster.
- Use memory efficiently.
- Enable higher-level graphical objects.

3.2 Pixel (Pel)

- Short for 'picture element'; also called a point or dot.
- The smallest picture unit / smallest addressable element on a display device.
- Closer pixel spacing → crisper, less jagged images; picture quality is directly proportional to pixel density.

3.3 Scan-Converting a Point

A pixel does not represent a true mathematical point but a small region that could theoretically contain infinite points. Scan-converting a point means illuminating the pixel that contains it.

3.4 Scan-Converting a Straight Line

A line is defined by two endpoints (x_1, y_1) and (x_2, y_2) ; its equation determines the (x, y) coordinates of all points that lie between them.

3.5 DDA (Digital Differential Analyzer) Algorithm

A simple, efficient line-drawing algorithm that uses incremental differences between the x- and y-coordinates of the endpoints to plot the line.

Steps

17. Input endpoints (x_1, y_1) and (x_2, y_2) .
18. Calculate $dx = x_2 - x_1$ and $dy = y_2 - y_1$.
19. Calculate slope $m = dy / dx$.
20. Set the initial point as (x_1, y_1) .
21. Loop through x, incrementing by 1 each time; compute $y = y_1 + m(x - x_1)$.
22. Plot the pixel at the calculated (x, y) .
23. Repeat until (x_2, y_2) is reached.

Pseudocode (increment form)

- $dx = X_1 - X_0$, $dy = Y_1 - Y_0$
- $steps = \max(|dx|, |dy|)$

- $X_{inc} = dx / steps$, $Y_{inc} = dy / steps$
- Loop 'steps' times: `put_pixel(round(X), round(Y)); X += Xinc; Y += Yinc`

Advantages

- Simple and easy to implement.
- Avoids operations with high time complexity like direct multiplication of the line equation.
- Faster than direct line-equation evaluation (no floating-point multiplication).

Disadvantages

- Uses rounding & floating-point arithmetic → relatively high time complexity.
- Orientation-dependent → poor endpoint accuracy.
- Limited floating-point precision → cumulative error over long lines.

3.6 Bresenham's Line Algorithm

An efficient line-drawing algorithm using only integer addition, subtraction, and multiplication — much faster than DDA. At each step it selects the next pixel with the least distance from the true line.

- Given $A(x_1, y_1)$ and $B(x_2, y_2)$, find all intermediate integer-coordinate pixels on line AB.
- Moves across the x-axis in unit steps; at each step chooses between (X_k+1, Y_k) and (X_k+1, Y_k+1) .
- Uses a decision parameter tracking accumulated slope error: once the error exceeds 0.5, y is incremented and the error is readjusted.

Exam tip: Be ready to trace through a worked Bresenham example step-by-step (compute decision parameter, dx , dy , and plot each pixel).

3.7 Scan Conversion of a Circle

- A circle is eight-way symmetric — the same shape in all four quadrants, with two symmetric octants per quadrant.
- Once one octant's points are calculated, the other seven follow by symmetry.

Two Standard Methods to Define a Circle (centered at origin)

- Polynomial (Cartesian) Method — algebraic equation relating (x, y) on the circumference to center (h, k) .
- Polar Coordinates Method — points defined as (r, θ) : r = radius/distance from center, θ = angle from positive x-axis.

Bresenham's Circle Algorithm

- Approximates and rasterizes points on a circle's circumference using integer arithmetic (avoids costly floating-point operations).
- Generates points from 90° to 45° and uses symmetry for the rest.

3.8 Scan Conversion of an Ellipse

Generates pixel coordinates approximating an ellipse's shape (rasterization). The Midpoint Ellipse Algorithm is the common method, iteratively deciding which pixels to include.

Midpoint Ellipse Algorithm — Outline

24. Initialize major/minor axes (a, b) ; start point $(x, y) = (0, b)$ — topmost point.
25. Calculate the initial decision parameter d from the ellipse equation.
26. Region 1: move along x, updating d ; choose $(x+1, y)$ or $(x+1, y-1)$.
27. Region 2 (once $d < 0$): move along the diagonal/y, choosing $(x+1, y-1)$ or $(x, y-1)$.

28. Plot each computed pixel and its symmetric points in the other three quadrants.

29. Repeat until the full ellipse is plotted.

Part 4: Filled Area Primitives

- Filled area primitives (solid shapes / filled polygons) are geometric objects with a closed set of vertices/edges whose interior is completely filled with color or texture.
- Fundamental for rendering complex scenes, 2D images, and GUIs.

Region Filling

- Boundary Fill algorithm — fills up to a defined boundary.
- Flood Fill algorithm — fills a connected interior region.

Part 5: 2D Transformations

5.1 Introduction

- Computer graphics lets us view an object from different angles (e.g., front/side/top elevation for architects) or resize maps/charts (cartography).
- Numeric image data stored in memory is modified using mathematical operations called Transformations.

Key Properties

- Each transformation is a single, uniquely named/symbolized entity.
- Transformations can be combined (concatenated): if A = translation and B = scaling, $C = AB$ combines both.

5.2 Two Views of Object Transformation

- Geometric Transformation — the object itself is transformed relative to a fixed coordinate system.
- Coordinate Transformation — the object stays fixed while the coordinate system is transformed relative to it.

5.3 Types of 2D Transformations

30. Translation

31. Scaling

32. Rotation

33. Reflection

34. Shearing

Translation

Moving an object from one position to another within the coordinate system — one of the most basic transformations.

Scaling

Alters an object's size while preserving its shape.

Rotation

- Changes the angle of the object about a pivot point (rotation point).
- Positive angle → counter-clockwise rotation; negative angle → clockwise rotation.
- Every point of the object rotates by the same angle.
- Line: rotate both endpoints, redraw between new endpoints.

- Polygon: shift every vertex by the same rotational angle.
- Curved line: reposition all points, redraw the curve.
- Circle: rotate about its center by the specified angle.
- Ellipse: rotate its major and minor axes by the desired angle.

Reflection

Produces a mirror image of an object (equivalent to a 180° flip about an axis). Types:

- 35. Reflection about the x-axis.
- 36. Reflection about the y-axis.
- 37. Reflection about an axis perpendicular to the xy-plane through the origin.
- 38. Reflection about the line $y = x$.

Shearing

- Distorts an object by shifting its vertices along one axis while the other axis stays unchanged.
- Changes the shape of the object (a 'sliding' of layers).
- Can be applied in one direction or two directions.

Part 6: Windows, Viewing & Navigation Techniques

6.1 Computer Graphics Window

A rectangular screen area where graphical output is rendered — the visible portion of the GUI/canvas.

- Display Area — the region where graphics are drawn.
- Coordinate System — usually origin top-left, x increases right, y increases downward.
- Resolution — number of pixels available; higher = sharper image.
- Viewport — the visible portion of the window actually showing graphics (can be smaller/larger than the window).
- Window Management — resize, maximize, minimize, close.
- Rendering Context — the canvas/API context (OpenGL, DirectX, HTML5 Canvas) used to draw.
- Interaction — mouse/keyboard/touch input to select, move, resize elements.

6.2 Zooming

Scales a pixel map (or portion of it) up or down at the pixel level based on user instruction; achieved by replicating pixels along successive scan lines.

6.3 Panning

Moves the viewpoint of a scene/image/document across the display area.

- Image Viewing — drag to explore large images that don't fit the window.
- 3D Scene Navigation — drag or use shortcuts to move the camera horizontally/vertically/along z-axis.
- Often combined with zooming and rotation for full navigational control.

6.4 Inking

The process of creating clean, finalized lines/outlines (digitally, e.g. Illustrator/Procreate, or traditionally with pens/markers) — refining and finalizing the visual elements of a design (illustrations, logos, typography).

Part 7: Clipping Techniques

Clipping limits the rendering of graphics primitives (lines, polygons, images) to a specific region of the screen/viewport — improves performance and creates cropping/masking effects.

Applications of Clipping

- Extracting the desired part of a scene.
- Identifying visible vs. invisible areas of a 3D object.
- Creating objects using solid modeling.
- Drawing operations.
- Pointing operations on an object.
- Deleting, copying, or moving part of an object.

Part 8: Pointing, Positioning & Interaction Techniques

8.1 Pointing vs. Positioning

- Pointing — referring to/selecting items already on the screen.
- Positioning — placing an item at a new screen location (from its old/current position).

The user indicates a screen position with an input device, which is then used to insert a symbol.

8.2 Pointing & Positioning Devices

Light Pen

A handheld pen-shaped device with a light-sensitive photodiode/phototransistor at the tip, connected by cable, used in early systems.

- Light Sensing — tip detects light emitted from the screen.
- Timing — sensor detects light when pressed against a specific screen position.
- Position Detection — hardware/software determine exact touch position from detection timing.
- Input Recognition — interpreted like a mouse click / touch input.

Mouse

- Pointing & Selection — move the cursor to interact with menus, icons, buttons.
- Positioning in 2D Graphics — precise drawing/positioning in tools like Photoshop/Illustrator.

Joystick

- Virtual Reality — point, grab, and manipulate 3D objects via VR controllers.
- 3D Modeling / CAD — alternative input for manipulating 3D objects and navigating complex models.

8.3 Elastic / Rubber-Band Technique

A technique for drawing geometric primitives (line, polyline, rectangle, circle, ellipse) where, as the user drags from a start point, the shape is dynamically previewed between the fixed endpoint and the current cursor position — like an elastic band stretching.

- De facto standard in GUI-based drawing applications.

8.4 Dragging

Moves an object from one screen position to another: the user selects the object, holds the mouse button, moves the cursor (object follows), and releases at the target position.

Part 9: Shading

Shading calculates the color and intensity of each pixel on a 2D/3D model to simulate how light interacts with the object's surface — creating a sense of realism and depth.

9.1 Illumination Models

- Ambient Lighting — a simple global light that uniformly illuminates all objects (base light level).
- Diffuse Lighting — models light scattered in many directions from a rough surface; gives a matte finish.
- Specular Lighting — simulates bright highlights on shiny objects; models directional reflection.

9.2 Shading Models

- Flat Shading — each polygon shaded based on the angle between the light source and the polygon's surface normal.

Gouraud Shading (Henri Gouraud, 1971)

- Calculates intensity/color at each vertex, then interpolates across the polygon surface.
- Steps: Vertex Intensity Calculation → Interpolation across the surface → efficient (computation only at vertices).

Phong Shading (Bui Tuong Phong, 1975)

- Calculates intensity/color at each pixel (not just vertices) for more realistic results.
- Steps: interpolate normal vectors across the surface (Normal Vector Interpolation) → calculate vertex normals from geometry → compute per-pixel intensity using interpolated normals, lighting, and material.
- Produces more realistic shading, especially on complex geometry, at higher computational cost than Gouraud.

9.3 Further Shading & Realism Techniques

- Texture Mapping — applies an image (texture) to a 3D surface for detail without added geometric complexity.
- Bump Mapping — simulates small bumps/wrinkles by perturbing surface normals.
- Environment Mapping — uses a precomputed texture to simulate reflective surfaces (mapping the environment onto the object).
- Shadow Mapping — renders the scene from the light's perspective to build a depth map determining shadowed areas.
- Ray Tracing — simulates light rays' paths to calculate reflections, refractions, and shadows for photorealism.
- Global Illumination — accounts for direct and indirect lighting (e.g., radiosity, photon mapping) for realism.
- Subsurface Scattering — simulates light penetrating/scattering inside translucent materials (skin, marble, milk).
- Physically Based Rendering (PBR) — mimics real physical light/material properties; standard in modern graphics engines.
- BRDF (Bidirectional Reflectance Distribution Function) — describes how light reflects off an opaque surface, relating incoming light direction and outgoing view direction to reflected radiance.

9.4 Properties of the Surface

- Reflectivity — how much light is reflected (mirror-like vs. matte); affects sharpness of reflections.
- Transparency — how much light passes through (glass/water vs. opaque wood/metal).
- Color — perceived from wavelengths reflected; influenced by pigmentation, chemical composition, texture.
- Texture — surface irregularities affecting light scatter (rough = matte, smooth = glossy).
- Normal — the surface's orientation vector relative to light/viewer; crucial for accurate lighting/shading.

9.5 Properties of the Illumination

- Intensity — brightness of light falling on the surface.
- Color — spectral composition of the light source affects perceived object color.
- Direction — directional (e.g., sunlight) vs. omnidirectional (e.g., bulb) light affects shadow distribution.
- Distance — farther light source → dimmer illumination (inverse square law).
- Ambient Lighting — indirect light bouncing off other surfaces, contributing to overall scene brightness.

Part 10: 3D in Computer Graphics

10.1 History of 3D

Early Developments (1960s–1980s)

- Roots trace to 1960s computer graphics research.
- Ivan Sutherland's Sketchpad (1963) — laid the foundation for interactive computer graphics, GUIs, and primitive 3D shapes.
- 1970s — first 3D wireframe models and rendering algorithms (e.g., hidden surface removal) emerged.

10.2 Introduction to 3D

- 3D transformations extend 2D transformations: 2D uses (x, y); 3D uses (x, y, z).
- 3D transformations: translation, scaling, rotation.
- 2D shows flat objects (bar charts, pie charts, graphs); 3D represents more natural objects, viewable in different sections/shapes.
- 3D rotation needs three factors — a component of three rotations, each along a Cartesian axis.

10.3 Core Concepts of 3D Graphics

- 3D Models — digital representations of objects/scenes built from points, lines, and polygons.
- Vertices, Edges, Polygons — vertices are points, edges connect them, polygons (usually triangles) form the model's faces.
- Textures — digital images applied to surfaces for color, pattern, detail.
- Lighting and Shading — simulate how light interacts with surfaces to create depth, shadow, and realism.

10.4 3D Geometry

- 3D coordinate system uses three axes: x, y, and z.
- Orientation can be right-handed or left-handed, affecting how coordinates are interpreted/visualized.

10.5 Producing Realism in 3D

- 3D objects are built using computer graphics; the 2D display technique for 3D objects is called projection.
- Realistic 3D scenes are essential for CAD — machine components, automobile bodies, aircraft parts.
- Realism requires understanding depth and perspective.

10.6 Types of 3D Graphics Based on Usage

- 39. 3D Modeling — creating 3D objects.
- 40. 3D Animation — making 3D objects move.
- 41. 3D Rendering — producing realistic images from 3D models.
- 42. 3D Simulation — virtual environments for training/testing.
- 43. 3D Visualization — presenting data, buildings, or products in 3D.

10.7 Types of 3D Models

- Wireframe Model — lines & vertices only, showing structure.
- Surface Model — represents outer surface without internal volume.
- Solid Model — complete representation including shape, volume, physical properties.
- Polygonal Model — built from connected polygons (triangles/quadrilaterals).

- Mesh Model — collection of vertices, edges, and faces defining shape.
- NURBS Model — mathematical curves/surfaces for smooth, precise shapes.
- Voxel Model — built from tiny cubes (voxels), like 3D pixels.
- Parametric Model — dimensions/features controlled by adjustable parameters.
- Procedural Model — auto-generated via algorithms and predefined rules.

10.8 3D Projections

- Parallel Projection
- Orthographic Projection
- Oblique Projection
- Isometric Projection
- Perspective Projection

Parallel Projection

- Lines of projection parallel to each other and perpendicular to the projection plane.
- No perspective distortion — objects keep relative proportions regardless of distance.
- Applications: technical drawings, engineering designs, architectural blueprints.

Orthographic Projection

- Lines perpendicular to the projection plane.
- True scale, no perspective distortion; typically top, front, and side views.
- Applications: engineering/architectural drawings, CAD.

Oblique Projection

- Lines parallel but at an angle (not perpendicular) to the plane.
- More pictorial than orthographic; partial depth effect; true scale kept along one axis (usually the front face).
- Applications: simple technical illustrations, early computer graphics, games.

Isometric Projection

- Lines parallel, forming equal 120° angles with the projection plane.
- No distortion along the axes — uniform scale; clear view of all three dimensions.
- Applications: video games (esp. older/simpler ones), technical/engineering drawings.

Perspective Projection

- Lines converge at one or more vanishing points.
- Mimics human eye perception — objects shrink with distance.
- Offers the most realistic representation of depth and space.
- Applications: realistic 3D rendering, video games, VR, simulations, architectural visualization.

Part 11: Rendering in 3D

Rendering is the process of generating an image from a 3D model using software or hardware — transforming geometric data, textures, lighting, and scene properties into a 2D image for display or print.

11.1 Evolution of Rendering (Timeline)

Wireframe Rendering (1960s)

Rendered images using simple geometric primitives (points, lines, polygons) — basic structure, no realistic lighting/shading.

Scanline Rendering (1970s)

Rasterized/rendered polygonal models by scanning each horizontal line and determining intersecting polygons, applying shading and visibility calculations.

Hidden Surface Removal (1970s)

Solved rendering of occluded/obscured objects. Key algorithms: the painter's algorithm and depth-buffering (z-buffering), determining which surfaces are visible from the viewer.

Phong Shading (1973)

Introduced by Bui Tuong Phong; simulated specular highlights and smooth shading, improving visual realism by accurately modeling light–surface interaction.

Ray Tracing (1980s)

Developed by researchers such as Turner Whitted and James Kajiya; traces rays from the camera through each pixel, computing intersections and simulating reflections, refractions, and shadows for photorealism.

Global Illumination (1980s–1990s)

Simulated realistic lighting — indirect illumination, color bleeding, soft shadows — via techniques like radiosity and photon mapping.

Texture Mapping (1980s–1990s)

Applied 2D images onto 3D surfaces, simulating materials like wood, metal, and fabric for richer visuals.

Real-Time Rendering (1990s–Present)

Hardware/GPU advances enabled interactive graphics at high frame rates — used in games, simulations, VR/AR.

Physically Based Rendering / PBR (2000s–Present)

Simulates light transport in a physically accurate way (energy conservation, reflectance, refraction) — widely used in film, architectural visualization, and product design.

11.2 Characteristics of Rendering

- Realism — closely resembles real-world scenes via accurate lighting, shadows, reflections, refractions, materials.
- Efficiency — high performance/scalability for complex scenes, enabling real-time/interactive rendering.
- Flexibility — options/parameters for lighting models, materials, anti-aliasing, post-processing.
- Accuracy — visually plausible results with minimal artifacts (aliasing, noise, banding).
- Interactivity — responsive feedback in games/simulations, often trading visual quality for speed.

11.3 Rendering Algorithms

Rasterization

Converts geometric primitives (points, lines, polygons) into pixels; determines which pixels each primitive covers and interpolates attributes (color, depth) across its surface.

Ray Tracing

Simulates light-ray behavior as they interact with scene objects; traces rays from camera through each pixel, computing intersections and lighting effects (reflection, refraction, shadow).

Shading Models

- Lambertian Reflection — simulates diffuse (matte) surfaces reflecting light equally in all directions.
- Phong Reflection — models specular highlights via reflection direction, surface normals, and viewer direction.
- Blinn-Phong Reflection — a more computationally efficient variant of Phong for specular highlights.

Global Illumination Algorithms

- Radiosity — computes light transfer between surfaces by solving an energy-balance equation system.
- Photon Mapping — a two-pass approach tracing photons from light sources, then estimating indirect lighting at each point.

Anti-Aliasing

- Supersampling — renders at higher resolution then downsamples to reduce aliasing.
- Multisampling — samples multiple points per pixel and averages results.
- Post-Processing — filters/blur applied after rendering to reduce aliasing artifacts.

Depth of Field & Motion Blur

- Depth of Field — blurs out-of-focus objects to simulate a camera lens, adding realism/depth.
- Motion Blur — simulates blur from movement of objects/camera during exposure, enhancing realism in dynamic scenes.

Part 12: Graphics APIs — DirectX & OpenGL

12.1 DirectX

A collection of APIs developed by Microsoft for multimedia tasks (especially gaming) on Windows, providing hardware-accelerated access to graphics and sound cards.

Components of DirectX

- DirectDraw — 2D graphics rendering, bypassing Windows GDI for performance.
- Direct3D — the 3D graphics component; rendering 3D models, textures, lighting, shaders/effects.
- DirectSound — hardware-accelerated audio playback (3D positional audio, environmental effects, mixing).
- DirectInput — handles input devices (keyboard, mouse, joystick, game controllers).
- DirectPlay — (deprecated) networking API for multiplayer gaming.
- DirectCompute — GPU computing API for general-purpose tasks (physics, image processing, ML).

Applications of DirectX

- Gaming; Multimedia Content Creation; VR/AR; Simulation & Training; Education/Edutainment; Architectural & Engineering Visualization; Digital Signage & Advertising; Medical Imaging & Visualization.

12.2 OpenGL (Open Graphics Library)

A cross-platform API for rendering 2D/3D vector graphics, developed by Silicon Graphics Inc. (SGI) in 1992, later managed as an open standard by the Khronos Group.

Rendering Pipeline Stages

44. Vertex Processing

- 45. Primitive Assembly
- 46. Rasterization
- 47. Fragment Processing
- 48. Framebuffer Operations

OpenGL Rendering Pipeline — Detailed Stages

- 1. Application Stage — object creation (3D models, textures, scenes) and scene management (placing objects, camera, lights).
- **2. Transformation Stage:**
 - Model Transformation — object coordinates: local (object) space → world space.
 - View Transformation — world space → camera (view) space, camera at origin.
 - Projection Transformation — 3D view → 2D plane (perspective or orthographic).
 - Clipping — discards objects/parts outside the view frustum.
 - Viewport Transformation — maps 2D coordinates to actual screen pixels.
- 3. Rasterization — converts 2D triangle vertices into pixels/fragments; fragment generation for each covered pixel (depth, color, texture coords).
- 4. Fragment Processing — Fragment Shader applies textures/color/lighting (incl. bump mapping, procedural textures); Depth Testing & Blending handle visibility and transparency.
- 5. Framebuffer Operations — final pixel colors written to the frame buffer; frame buffer displayed on screen as the rendered image.

Key OpenGL Features

- Cross-Platform — runs on Windows, macOS, Linux, Android, iOS.
- Hardware Acceleration — leverages GPU capabilities.
- Texture Mapping — applies images/patterns to 3D surfaces.
- Extensibility — extension mechanisms for new hardware capabilities/techniques.
- Shading Language (GLSL) — for programming custom vertex and fragment shaders.
- Buffer Objects — efficient storage/management of vertex data, textures, resources.
- Transformation & Projection Matrices — manipulate/transform 3D geometric objects.
- Lighting & Material Properties — simulate lighting; define color, shininess, transparency.
- Framebuffer Objects (FBOs) — enable off-screen rendering (render-to-texture, post-processing).

Applications of OpenGL

- Video Games; CAD Software; Animation Software; VR/AR; Scientific Visualization; GIS; Digital Content Creation (DCC) Tools; Educational Software; Medical Imaging & Visualization; Simulation & Training.

Quick Revision Summary — High-Yield Points

Line-Drawing Algorithms

- DDA — simple, incremental, uses floating point → rounding errors, orientation-dependent.
- Bresenham's — integer-only arithmetic → faster, more accurate, industry standard.

Shading Models Comparison

- Flat Shading — one shade per polygon (fastest, least realistic).
- Gouraud Shading — interpolates vertex colors across the polygon (fast, moderate realism).
- Phong Shading — interpolates surface normals, computes lighting per pixel (slowest, most realistic).

Projection Types Comparison

- Parallel/Orthographic — true scale, no distortion, used in engineering drawings.
- Oblique — partial depth effect, true scale on one face.
- Isometric — equal 120° angles, uniform scale, all 3 dimensions visible.
- Perspective — vanishing point(s), most realistic, used in games/VR.

CRT Color Methods

- Beam Penetration — random-scan, 4 colors only, cheap.
- Shadow-Mask — full RGB range, used in modern-era CRT color monitors.

DirectX vs OpenGL

- DirectX — Microsoft, Windows-only, tightly integrated with Windows hardware (gaming focus).
- OpenGL — Khronos Group, cross-platform, open standard.

Good luck with your exam! Review the algorithm steps (DDA, Bresenham, Midpoint Ellipse) and be ready to trace through worked examples, as these are common exam question types.